

NEBULAOPS

v21.2

Piattaforma DevOps Enterprise Cloud-Native

Documentazione Ufficiale — Manuale Utente · Architettura · Operations ·
Troubleshooting

Angular 18	Spring Boot 3.3	Java 21	Go 1.23	Python 3.12
Docker Compose	Kubernetes	Helm 3	ArgoCD	Terraform
MongoDB 7	RabbitMQ 3.13	Redis 7	Prometheus	Grafana 11.1

Peyman Eshghi Malayeri · MIT License · Maggio 2026 · v21.2.0

Indice dei Contenuti

1. Introduzione e Vision Platform

- Cos'è NebulaOps · Filosofia Cloud-Native · Principi architetturali
- Confronto con soluzioni SaaS commerciali · Target audience
- Requisiti di sistema · Compatibilità WSL2 e Linux nativo

2. Architettura della Piattaforma

- Diagramma architetturale completo
- Gateway Service — cuore del sistema e pattern Proxy
- Catalogo completo dei 15 servizi con ruoli e responsabilità
- Flussi dati: Docker Desktop · OpenLens · Tasks · AI OPS
- Configurazione centralizzata: platform.yml come Single Source of Truth

3. Installazione e Primo Avvio

- Prerequisiti dettagliati WSL2 / Linux nativo
- Procedura di installazione passo-passo
- Configurazione del file .wslconfig per performance ottimali
- Verifica completa dell'installazione · Script di gestione

4. Manuale Utente — Interfaccia Completa

- Login e navigazione · Struttura dell'interfaccia Angular 18
- OpenLens: Kubernetes Control Plane (13 sezioni)
- Docker Desktop: Container Runtime via Unix Socket API
- Platform Dashboard: 9 moduli operativi avanzati
- AI OPS: Grafico 3D dipendenze · RCA · Auto-fix

5. Helm — Package Manager Kubernetes

- Concetti fondamentali: Chart · Release · Values · Templating
- Struttura del Helm Chart NebulaOps · Deploy e upgrade
- Best practice produzione · Override values per ambiente

6. ArgoCD — GitOps Continuo

- Paradigma GitOps: Git come source of truth per il cluster
- Application · Sync · Drift · Auto-heal · Health assessment
- Integrazione con NebulaOps · Flusso GitOps completo

7. Stack di Osservabilità

- I tre pilastri: Metriche · Log · Trace distribuiti
- Prometheus: scraping, PromQL, alerting
- Loki: log aggregation, LogQL · Tempo: distributed tracing
- Grafana: dashboard unificata, correlazione segnali
- OpenTelemetry Collector: routing centralizzato telemetria

8. CI/CD e Deploy su NebulaOps

- Pipeline GitHub Actions: 7 stage dalla commit al deploy

- Integrazione GitLab CI · Deploy con kubectl e Helm
- Il progetto demo nebulaops-demo: Spring Boot end-to-end
- Strategie di deploy: RollingUpdate · Blue/Green · Canary

9. DevSecOps e Sicurezza

- Trivy: vulnerability scanning immagini Docker
- Secret management · SBOM · Compliance baseline
- Non-root containers · Security context Kubernetes

10. Terraform e Infrastructure as Code

- Terraform Studio integrato in NebulaOps
- Moduli disponibili · Plan/Apply workflow · Cost estimation

11. Troubleshooting e Problemi Noti

- Errori 502 gateway: diagnosi e fix
- Docker Desktop vuoto: causa root e soluzione definitiva
- OpenLens senza dati: kubectl vs Docker socket fallback
- MongoDB · RabbitMQ · Redis: problemi comuni
- Performance e gestione memoria WSL2

12. Riferimento API · Estensione della Piattaforma

- Tutti gli endpoint gateway con parametri e response shape
- API del progetto demo nebulaops-demo
- Come aggiungere un nuovo microservizio in 7 step

CAPITOLO 1

Introduzione e Vision Platform

Cos'è NebulaOps, perché esiste, a chi serve

1.1 Cos'è NebulaOps

NebulaOps è una piattaforma DevOps enterprise **local-first** che centralizza il controllo dell'intero ciclo di vita dell'infrastruttura in un'unica interfaccia Angular 18 dark-mode 3D. Non è un semplice dashboard: è un *cockpit operativo completo* che integra nativamente container Docker, cluster Kubernetes, pipeline CI/CD, monitoring distribuito, sicurezza DevSecOps e Infrastructure as Code — eliminando il bisogno di passare tra decine di strumenti separati durante le operazioni quotidiane.

La piattaforma gira interamente in locale su WSL2 o Linux nativo tramite Docker Compose, orchestrando oltre **25 container**: 12 microservizi Spring Boot, 2 servizi Go ad alte prestazioni, 1 AI engine Python FastAPI, e un intero stack di osservabilità professionale (Prometheus, Loki, Tempo, Grafana, OpenTelemetry). Nessun cloud obbligatorio, nessun costo di infrastruttura per lo sviluppo e il testing locale.

1.2 Filosofia Cloud-Native e Principi Architettureali

NebulaOps abbraccia integralmente i **Twelve-Factor App** e i principi delle applicazioni cloud-native definiti dalla CNCF (Cloud Native Computing Foundation). Ogni componente è progettato per essere stateless, containerizzato, configurabile via environment variables, e scalabile orizzontalmente senza modifiche al codice.

Microservizi indipendenti: Ogni dominio di business (autenticazione, task management, pipeline, AI ops, sicurezza) è un microservizio separato con il proprio ciclo di deploy, database logico e team di sviluppo autonomo. Nessun accoppiamento diretto tra servizi — comunicazione solo via API REST o messaggi asincroni.

API-first e Gateway Pattern: Il gateway-service è l'unico punto di ingresso per il frontend. Implementa il pattern BFF (Backend For Frontend) adattando le risposte alle necessità specifiche dell'Angular SPA. Il frontend non conosce gli indirizzi interni dei microservizi.

Single Source of Truth per la configurazione: config/platform.yml contiene tutti i service URL, le porte e le route API. Ogni altro file di configurazione è derivato da questa sorgente canonica, eliminando la desincronizzazione tra ambienti.

Osservabilità by default: Ogni microservizio Spring Boot espone /actuator/prometheus, /actuator/health/liveness e /actuator/health/readiness. I trace distribuiti viaggiano automaticamente via OpenTelemetry verso Tempo. Nessuna modifica al codice applicativo richiesta.

Graceful degradation: Quando un servizio downstream non risponde, il gateway restituisce strutture di fallback con live:false invece di propagare errori HTTP 500/502. Il frontend gestisce gracefully l'assenza di dati live mostrando stati appropriati.

GitOps come standard: L'intero stato dell'infrastruttura è descritto in codice versionato: Helm chart, manifest Kubernetes, moduli Terraform. ArgoCD garantisce che il cluster converga sempre verso lo stato dichiarato in Git, con reconciliation automatica.

Security by design: Container non-root, scan automatici Trivy in CI/CD, secret management via K8s Secrets, NetworkPolicy per isolamento namespace, SBOM generazione per ogni release.

1.3 Confronto con Soluzioni SaaS Commerciali

Funzionalità	NebulaOps v21.2	Datadog	ArgoCD Cloud	Rancher
Kubernetes Control Plane	✓ Integrato	Parziale	✓ Nativo	✓ Nativo
Docker Desktop UI	✓ Via Unix Socket	✗	✗	Parziale
CI/CD Pipeline View	✓ Integrato	✓	✗	Parziale
AI OPS / RCA	✓ AI Engine Python	✓ (costoso)	✗	✗
Terraform Studio	✓ Integrato	✗	✗	✗
GitOps (ArgoCD)	✓ Wrapper nativo	✗	✓	✓
Observability Stack	✓ Prometheus+Loki+Tempo	✓ (SaaS \$\$\$)	✗	Parziale
Costo infra locale	€0 (Docker Compose)	\$\$\$	\$\$\$	\$ Self-hosted
Privacy dati	✓ Totale (locale)	Cloud	Cloud	Self-hosted

1.4 Requisiti di Sistema

Sistema Operativo	Windows 11 con WSL2 (Ubuntu 22.04+) oppure Linux nativo (Ubuntu 22.04 / Debian 12)
RAM	Minimo 16 GB raccomandati — lo stack completo usa ~6-8 GB. Con 32 GB le performance sono ottimali.
CPU	8 core fisici — 25+ container si avviano in parallelo durante lo startup.
Storage	25 GB liberi — ~8 GB immagini Docker + volumi dati MongoDB/Redis/Grafana.
Docker	Docker Desktop ≥ 4.25 con WSL2 backend abilitato, oppure Docker Engine nativo su Linux.
Java (build)	JDK 21 richiesto solo se si buildano i servizi fuori Docker. Non richiesto per il runtime.
kubectl	v1.29+ raccomandato per connessione a cluster Kubernetes reale. Opzionale — senza, il gateway usa il Docker socket come fallback.
Porte richieste	4200, 8080-8099, 3000, 9090, 3100, 3200, 27017, 5672, 15672, 6379, 8088-8089, 4318

■ ATTENZIONE

Su WSL2: assicurarsi che Docker Desktop → Settings → Resources → WSL Integration sia abilitato per la distribuzione Ubuntu. Senza questo, i container non vedono il socket Docker dell'host e `/api/runtime/*` restituirà liste vuote.

CAPITOLO 2

Architettura della Piattaforma

Topologia completa, pattern di comunicazione, responsabilità di ogni componente

2.1 Diagramma Architetturale Completo

ARCHITETTURA COMPLETA — NEBULAOPS v21.2

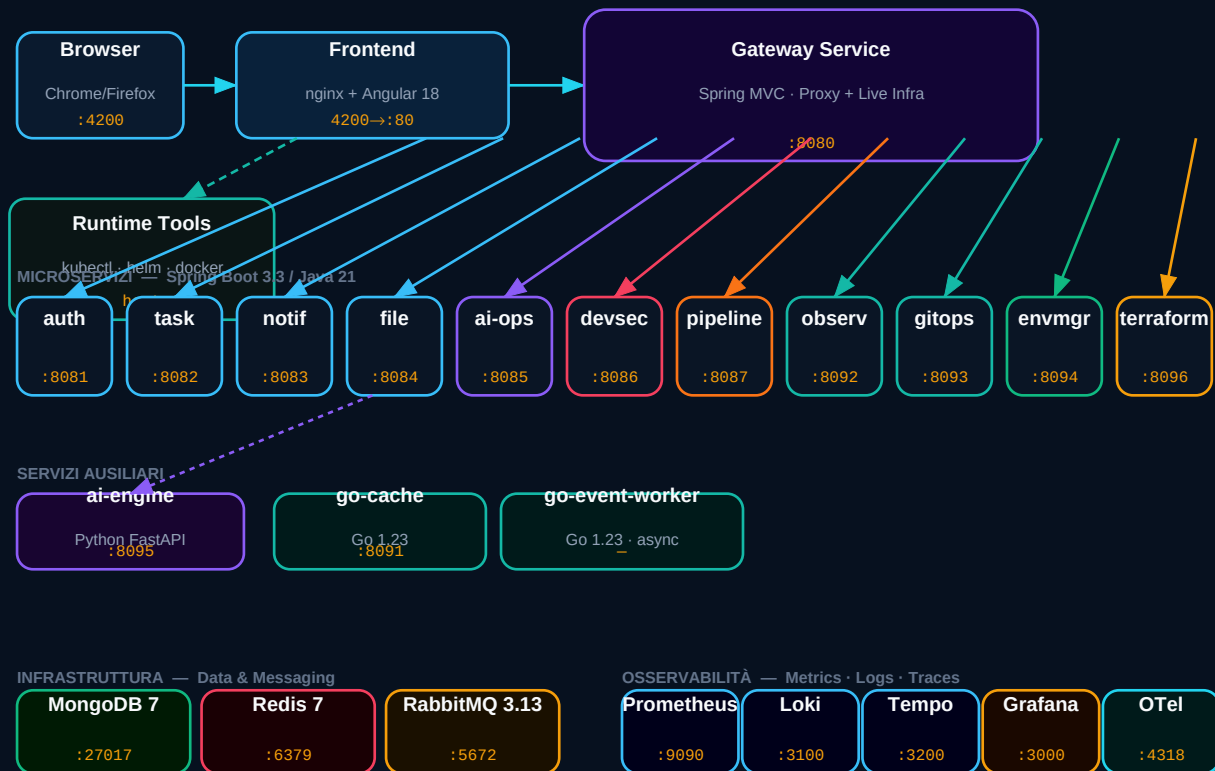


Fig. 1 — Architettura completa NebulaOps v21.2: dal browser ai microservizi all'infrastruttura

2.2 Gateway Service — Il Cuore del Sistema

Il **gateway-service** (Spring Boot MVC, :8080) è il componente più critico dell'intera piattaforma. Implementa tre pattern distinti in un unico servizio:

Pattern 1 — API Gateway e BFF (Backend For Frontend)

Riceve tutte le chiamate dal frontend Angular e le instradata ai microservizi downstream tramite RestTemplate con JdkClientHttpRequestFactory (necessario per supportare il metodo HTTP PATCH che la JVM gestisce nativamente). Il ProxyController mantiene un mapping da route pubblica a servizio interno, con fallback strutturati quando il servizio non è disponibile.

Pattern 2 — Docker Socket Client

Invece di eseguire il binary docker tramite ProcessBuilder — approccio fragile perché il binary potrebbe essere uno snap, un wrapper o non compatibile con l'ambiente container — il gateway comunica direttamente con /var/run/docker.sock via HTTP Unix socket usando UnixDomainSocketAddress (Java 16+). Questo bypassa completamente il CLI e chiama le Docker Engine API native: GET /containers/json?all=true, GET /images/json, GET /volumes. Il risultato viene normalizzato da RuntimeOpsController nei campi che Angular si aspetta.

Pattern 3 — Kubernetes Fallback

Quando kubectl non è configurato (nessun cluster disponibile), KubernetesOpsController.snapshot() costruisce una vista K8s-style dai container Docker reali. Ogni container diventa un Deployment + Pod + Service nella risposta, permettendo alle sezioni OpenLens di mostrare dati reali anche in ambiente locale senza Kubernetes.

2.3 Catalogo Completo dei Servizi

Servizio	Tecnologia	Porta	Responsabilità
gateway-service	Spring MVC	:8080	API gateway, proxy BFF, Docker socket client, K8s fallback builder
auth-service	Spring Boot 3.3	:8081	Registrazione utenti, autenticazione dev-mode, gestione sessioni
task-service	Spring Boot 3.3	:8082	CRUD task, stati BACKLOG→DONE, pubblica eventi RabbitMQ su task.created/updated
notification-service	Spring Boot 3.3	:8083	Consumer RabbitMQ, persiste notifiche su MongoDB, delivery real-time
file-service	Spring Boot 3.3	:8084	Metadata file, astrazione storage, presigned URL per accesso diretto
ai-ops-service	Spring Boot 3.3	:8085	Wrapper verso ai-engine, orchestrazione analisi RCA e autofix
devsecops-service	Spring Boot 3.3	:8086	Scan Trivy immagini via Docker socket, secret scanning, SBOM generation
pipeline-engine-service	Spring Boot 3.3	:8087	Stato pipeline CI/CD, cronologia run, stage tracking, trigger manuale
observability-service	Spring Boot 3.3	:8092	Aggregatore Prometheus+Loki+Tempo, audit events, SLO tracking
gitops-control-service	Spring Boot 3.3	:8093	Wrapper ArgoCD API, sync, drift detection, health assessment Application

Servizio	Tecnologia	Porta	Responsabilità
environment-manager-service	Spring Boot 3.3	:8094	Lifecycle namespace Kubernetes, ambienti dev/staging/prod, quota management
terraform-studio-service	Spring Boot 3.3	:8096	Orchestrazione plan/apply Terraform, cost estimation, state management
ai-engine	Python FastAPI 3.12	:8095	Anomaly detection ML, clustering log, Root Cause Analysis inference
go-cache-service	Go 1.23	:8091	Cache layer ad alte prestazioni su Redis, TTL management, invalidation
go-event-worker	Go 1.23	interno	Consumer background RabbitMQ, processing asincrono eventi, retry logic

2.4 Infrastruttura di Supporto

Componente	Versione	Porta	Ruolo
MongoDB	7.0	:27017	Database primario — collezioni per task, notifiche, file metadata, log
Redis	7 Alpine	:6379	Cache in-memory, session state, hot lookup acceleration
RabbitMQ	3.13 Management	:5672 / :15672	Message broker — exchange nebulaops.*, routing eventi asincroni
Prometheus	v2.53.0	:9090	Scraping metriche /actuator/prometheus ogni 15s, storage time-series
Loki	2.9.10	:3100	Aggregazione log container, query LogQL
Tempo	2.5.0	:3200	Backend distributed tracing, query TraceQL
Grafana	11.1.0	:3000	Dashboard unificata — datasource: Prometheus, Loki, Tempo
OTel Collector	0.104.0	:4318 / :4317	Riceve e routing telemetria OTLP HTTP/gRPC
Mongo Express	1.0.2	:8088	UI MongoDB — admin/admin
Redis Commander	latest	:8089	UI Redis — admin/admin

2.5 Flusso Dati: Docker Desktop

```
# Frontend Angular → API gateway → Docker Engine API
Angular: loadDocker() → forkJoin([containers, images, volumes])
HTTP GET /api/runtime/docker/containers
nginx proxy_pass → gateway-service:8080
RuntimeOpsController.containers()
DockerRuntimeService.containers()
```

```
DockerSocketClient → UnixDomainSocketAddress("/var/run/docker.sock")
HTTP/1.0 GET /containers/json?all=true
Response: [{Id, Names[], Image, State, Ports[], NetworkSettings...}]
RuntimeOpsController.normalizeContainer(raw)
→ {id, name, image, status, ports, network, logs}
signal dockerContainers.set(containers) → UI aggiornata
```

2.6 Configurazione Centralizzata: platform.yml

Tutti i service URL, le porte e le route API derivano da un unico file: config/platform.yml. Le modifiche in questo file devono essere propagate in tre punti: application.yml del gateway (proxy targets), api.config.ts del frontend (typed URL constants), docker-compose.yml (environment variables per container).

■ ATTENZIONE

Mai modificare URL di servizi direttamente in application.yml o api.config.ts senza aggiornare platform.yml. La desincronizzazione tra questi file è la causa più comune di errori 502 che non hanno una spiegazione ovvia.

CAPITOLO 3

Installazione e Primo Avvio

Procedura completa da zero a piattaforma operativa

3.1 Configurazione WSL2 Ottimale

Prima di avviare NebulaOps su Windows, è essenziale configurare correttamente WSL2 per garantire risorse sufficienti allo stack completo. Creare o modificare il file %USERPROFILE%\wslconfig con i seguenti parametri:

```
# %USERPROFILE%\wslconfig - Configurazione WSL2

[wsl2]

memory=16GB # Minimo 16GB - il stack usa 6-8GB al regime
processors=8 # Numero di vCPU - uguale ai core fisici
swap=4GB # Swap di sicurezza
localhostForwarding=true

# Dopo la modifica: riavviare WSL2 da PowerShell
wsl --shutdown

# Riaprire Ubuntu e verificare
free -h # → Mem: should show ~16G
nproc # → should show 8
```

■ CRITICO

Docker Desktop → Settings → Resources → WSL Integration: abilitare il toggle per la distribuzione Ubuntu. Senza questo passaggio il socket Docker non sarà accessibile ai container e tutte le sezioni Docker Desktop + OpenLens mostreranno dati vuoti.

3.2 Installazione Strumenti di Base

```
# Su Ubuntu WSL2 o Linux nativo

# Docker Engine (se non si usa Docker Desktop)
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER && newgrp docker

# kubectl
curl -fsSL "https://dl.k8s.io/release/$(curl -sL https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl" -o kubectl
chmod +x kubectl && sudo mv kubectl /usr/local/bin/

# Helm
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

# kind (cluster Kubernetes locale)
```

```
go install sigs.k8s.io/kind@latest
# oppure: curl -Lo ./kind
https://github.com/kubernetes-sigs/kind/releases/latest/download/kind-linux-amd64

# Verifica versioni

docker --version # Docker version 25.0+
docker compose version # Docker Compose version v2.24+
kubectl version --client
helm version
```

3.3 Avvio NebulaOps

```
# 1. Estrai lo ZIP in una directory di lavoro
cd /mnt/d/workspace/portfolio/nebulaops-v21.2

# 2. Prima volta: build completo (~8-12 minuti con download dipendenze Maven)
./scripts/wsl/start.sh

# Con rebuild forzato del gateway (obbligatorio dopo modifiche al BE)
./scripts/wsl/start.sh --rebuild-gateway

# 3. Monitora l'avvio – attendi che i servizi critici siano (healthy)
docker compose ps

# mongodb → (healthy) dopo ~30s
# rabbitmq → (healthy) dopo ~20s
# redis → (healthy) dopo ~5s
# gateway-service → Up dopo ~45s dal build

# 4. Verifica health di tutti i servizi
./scripts/wsl/health.sh

# 5. Apri la piattaforma
xdg-open http://localhost:4200
```

■ INFO

La prima esecuzione scarica ~800MB di dipendenze Maven (layer cache Docker). Dalla seconda esecuzione in poi, grazie al layer caching, il tempo scende a ~2-3 minuti. Garantire connessione internet durante il primo avvio.

3.4 Verifica Completa dell'Installazione

```
# Gateway health check – risposta attesa: {"status":"UP","version":"21.2"}
curl -s http://localhost:8080/api/health | python3 -m json.tool

# Verifica Docker socket dal gateway
curl -s http://localhost:8080/api/runtime/docker/containers | python3 -m json.tool | head -30

# → Array di container con [{id, name, image, status, ports}...]

# Verifica microservizi critici
for port in 8081 8082 8083 8085 8086 8087; do
```

```
echo -n "Port $port: "
curl -sf http://localhost:$port/actuator/health | python3 -c "import sys,json;
d=json.load(sys.stdin); print(d['status'])"
done

# Verifica stack osservabilità
curl -s http://localhost:9090/-/healthy && echo "Prometheus OK"
curl -s http://localhost:3000/api/health | python3 -m json.tool
# → {"database":"ok"}

# Verifica snapshot Kubernetes
curl -s http://localhost:8080/api/kubernetes/snapshot | \
python3 -c "import sys,json; d=json.load(sys.stdin); print(f'resources: {len(d[chr(114)+c
hr(101)+chr(115)+chr(111)+chr(117)+chr(114)+chr(99)+chr(101)+chr(115)]}, live:
{d[chr(108)+chr(105)+chr(118)+chr(101)]})'"
```

3.5 Script di Gestione Operativa

Script	Descrizione	Quando usarlo
./scripts/wsl/start.sh	Avvio completo stack. --rebuild-gateway per no-cache gateway	Primo avvio, dopo modifiche al codice
./scripts/wsl/restart-gateway.sh	Rebuild + restart solo gateway con --no-cache	Dopo modifiche a controller Java o proxy config
./scripts/wsl/stop.sh	Stop tutti i container. I volumi dati vengono preservati	Fine sessione di lavoro
./scripts/wsl/health.sh	Curl su tutti gli actuator, output con semafori colorati	Diagnosi problemi, verifica post-avvio
./scripts/wsl/logs.sh [svc]	Stream log di un servizio specifico in tempo reale	Debug runtime di un servizio
./scripts/wsl/diagnose-runtime.sh	Diagnosi completa: socket, kubectl, helm, porte	Troubleshooting avanzato

CAPITOLO 4

Manuale Utente — Interfaccia Completa

Guida operativa sezione per sezione

4.1 Accesso e Struttura dell'Interfaccia

Naviga su `http://localhost:4200`. Credenziali default: `admin / admin`. L'interfaccia Angular 18 è costruita come Single Page Application con Angular Signals per la reattività — ogni aggiornamento di stato è immediato senza refresh della pagina.

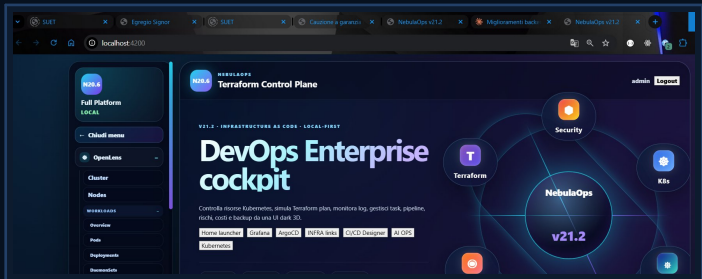


Fig. 2 — Homepage NebulaOps v21.2: sidebar OpenLens+Docker, header con tab principali, content area centrale

L'interfaccia è divisa in tre zone: **sidebar sinistra** collassabile con la navigazione gerarchica (OpenLens → Docker Desktop → Platform), **header topbar** con le tab principali (OVERVIEW, KUBERNETES, CONTAINERS, ecc.), **content area** centrale che cambia completamente in base alla sezione attiva. Lo stato di connessione della piattaforma è visibile nel badge in alto a sinistra (LOCAL / SYNCING / CONNECTED / ERROR).

4.2 OpenLens — Kubernetes Control Plane

La sezione OpenLens replica l'esperienza del client Lens/OpenLens direttamente nell'interfaccia. Quando un cluster Kubernetes reale è configurato (kubeconfig montato in `.kube/config`), mostra le risorse live via kubectl. In assenza di cluster, costruisce una vista K8s-style dai container Docker reali, garantendo dati sempre visibili.

Sezione	Dati mostrati	Azioni disponibili
Cluster	Metriche aggregate: servizi attivi, CPU%, memoria, network I/O	Refresh, Export metrics
Nodes	Nodi con status Ready/NotReady, versione K8s, OS, capacità	Describe node, Cordon, Uncordon
Pods	Tutti i pod con namespace, stato (Running/Pending/Failed/Stopped), immagine, età	Logs, Exec, Delete, Port-forward

Sezione	Dati mostrati	Azioni disponibili
Deployments	Replicas desired vs available, strategia, immagine, età	Scale, Restart Rolling, Edit YAML
StatefulSets	StatefulSet con storage associato, ordinal, ready	Scale, Restart, Describe
DaemonSets	DaemonSet per nodo con desired/current/ready	Describe, Delete
Services	ClusterIP/NodePort/LoadBalancer con selector e porte	Describe, Port-forward
Ingresses	Regole routing HTTP con hostname, path, backend, TLS	Describe, Edit
Namespaces	Lista namespace con status Active/Terminating	Create, Delete, Set default
Events	Log eventi K8s in real-time — Warning evidenziati in arancione	Filter by type, Export
Config/Secrets	ConfigMap e Secret (valori offuscati automaticamente)	View, Edit, Delete
Storage	PersistentVolume, PVC, StorageClass con capacità e binding	Describe, Delete PVC
Port Forwarding	Form per configurare kubectl port-forward dall'UI	Start, Stop forwarding

4.3 Docker Desktop — Container Runtime

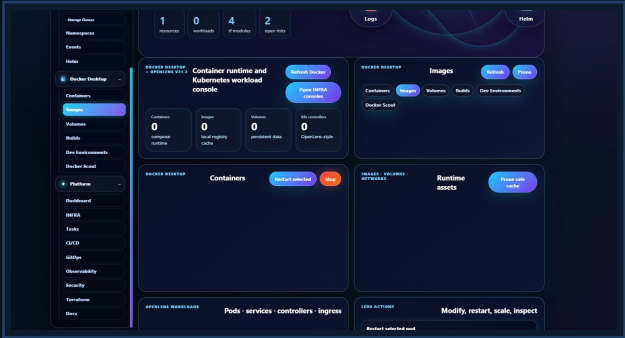


Fig. 3 — Docker Desktop: pannello Container Runtime con Images, Volumes e Runtime Assets

La sezione Docker Desktop usa il **Docker Engine API** via Unix socket (/var/run/docker.sock) senza dipendere dal binary docker CLI. Questo approccio funziona in qualsiasi ambiente container indipendentemente dal binary docker presente sull'host (snap, wrapper, ecc.). I dati si aggiornano ad ogni cambio di sezione e al click su Refresh.

Sezione	Contenuto	Azioni principali
Containers	Lista tutti i container (running + stopped). Per ogni container: nome, immagine, stato, CPU%, memoria, porte mappate.	Restart selected, Stop, Open INFRA consoles
Images	Immagini locali con repository, tag, dimensione, numero vulnerabilità Trivy (0 = verde).	Prune dangling, Refresh, Pull
Volumes	Volumi con nome, driver (local/nfs), mountpoint, scope.	Prune safe cache (rimuove non usati)
Builds	Cronologia build Docker Buildx con esito e durata.	View logs, Rebuild
Dev Environments	Ambienti di sviluppo containerizzati Docker.	Create, Start, Stop
Docker Scout	Analisi sicurezza SBOM via Docker Scout CLI.	Scan image, View CVEs

4.4 Platform — Moduli Operativi Avanzati

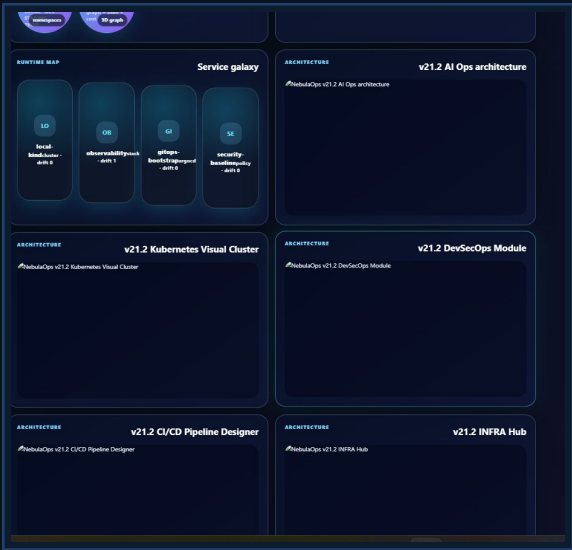


Fig. 4 — Service Galaxy e pannelli Architecture con diagrammi SVG interattivi della piattaforma

Modulo	Descrizione operativa
Dashboard	Panoramica: task attivi, open risks, TF modules, resources. Grafico Service Galaxy 3D con nodi cliccabili per ogni ambiente (local, observability, gitops, security).
INFRA Hub	Mappa interattiva 3D dell'infrastruttura. Link diretti a MongoDB Express (:8088), RabbitMQ Management (:15672), Redis Commander (:8089), Grafana (:3000), Prometheus (:9090).
Tasks (Kanban)	Board drag-and-drop con colonne BACKLOG→TODO→IN PROGRESS→REVIEW→DONE. Ogni task ha titolo, descrizione, priorità, assegnatario. Task persistiti su MongoDB via task-service, eventi su RabbitMQ.

Modulo	Descrizione operativa
CI/CD Designer	Visualizzatore pipeline con stage Source→Build→Test→Security→Deploy. Per ogni stage: stato (pass/fail/running), durata, log dell'esecuzione.
GitOps	Tabella Application Set ArgoCD con nome, namespace, stato sync, drift (differenze Git vs cluster). Pulsante Sync per forzare riconciliazione immediata.
Observability	Dashboard con metriche Prometheus aggregate, log Loki degli ultimi 15 min, trace Tempo per richiesta. Service health gauge, error rate, P95 latency per servizio.
Security	Vulnerability scan Trivy per immagini, secret scanning nei repository, compliance baseline PSP/NetworkPolicy, SBOM per release.
Terraform Studio	Editor piano/apply Terraform con stima costo per modulo. Output variables visualizzati dopo apply. Stato moduli attivi con versione e provider.
Docs	Diagrammi SVG architetturali interattivi: Full Platform, AI OPS, K8s Cluster, CI/CD Pipeline, DevSecOps, GitOps, INFRA Hub, Terraform Studio, Observability Stack.

4.5 AI OPS — Intelligenza Operativa



Fig. 5 — AI OPS: grafico 3D dipendenze servizi con nodi colorati per stato health (verde=ok, arancio=warn, rosso=critical)

Il modulo AI OPS è il centro diagnostico intelligente della piattaforma. Il grafico 3D delle dipendenze mostra i servizi come nodi posizionati in uno spazio tridimensionale — la posizione Z indica la criticità, il colore lo stato di salute. Cliccando su un nodo si ottengono i dettagli e le azioni disponibili.

Funzionalità	Descrizione
Grafico dipendenze 3D	Nodi: Frontend, Gateway, Tasks, MongoDB, Notify — con coordinate X/Y/Z e colore stato. Linee di connessione animate mostrano i flussi di traffico.
Analyze (RCA)	Invia un prompt all'AI engine (Python FastAPI) che analizza log e metriche e restituisce suggerimenti di Root Cause Analysis con livello di confidenza.
Auto-fix	Applica automaticamente i fix suggeriti dall'AI — restart servizi, scaling repliche, aggiornamento configurazioni.
Realtime Event Timeline	Log degli eventi infrastrutturali in ordine cronologico con livello (INFO/WARN/ERROR) e servizio di origine.
Incident Propagation	Grafo di impatto cascata — mostra quali servizi sono affected da un'anomalia nel servizio selezionato.

CAPITOLO 5

Helm — Package Manager Kubernetes

Chart, Release, Templating, Upgrade, Rollback

5.1 Cos'è Helm e Perché Usarlo

Helm è il package manager standard per Kubernetes, mantenuto dalla CNCF. Risolve il problema della gestione di applicazioni Kubernetes complesse che richiedono decine di manifest YAML correlati (Deployment, Service, Ingress, ConfigMap, Secret, HPA, ecc.): Helm li raggruppa in un *Chart* parametrizzabile tramite file *values.yaml*, permettendo di installare l'intera applicazione con un singolo comando e di gestirne il ciclo di vita (upgrade, rollback, uninstall) in modo atomico.

Concetto Helm	Definizione
Chart	Pacchetto Helm: collezione di template YAML + values.yaml + Chart.yaml. Equivalente a un pacchetto npm o apt.
Release	Istanza di un Chart installato in un cluster. Stesso Chart può avere release diverse (dev, staging, prod).
Values	File YAML con i parametri di configurazione: immagine, repliche, risorse, environment variables. Sovrascrivibili a runtime.
Repository	Collezione di Chart pubblicati — Artifact Hub, Bitnami, charts interni.
Template	File YAML con Helm templating (Go templates) per parametrizzazione.
Hooks	Script eseguiti in fasi specifiche del ciclo di vita (pre-install, post-upgrade, ecc.)
Rollback	Torna alla revision precedente di una release in caso di problemi.

5.2 Struttura del Helm Chart NebulaOps-Demo

```
helm/nebulaops-demo/
■■■ Chart.yaml # Metadati: name, version, appVersion, description
■■■ values.yaml # Parametri default: image, replicas, resources, env
■■■ templates/
■■■ deployment.yaml # Deployment parametrizzato con {{ .Values.image.tag }}
■■■ service.yaml # Service con porta da {{ .Values.service.port }}

# values.yaml – override a runtime
replicaCount: 2
image:
repository: nebulaops-demo
tag: latest
```

```
pullPolicy: IfNotPresent
resources:
  requests: {cpu: 250m, memory: 256Mi}
  limits: {cpu: 500m, memory: 512Mi}
autoscaling:
  enabled: true
  minReplicas: 2
  maxReplicas: 6
  cpuTargetUtilization: 60
```

5.3 Comandi Helm Essenziali

```
# Installazione (prima volta)
helm install nebulaops-demo ./helm/nebulaops-demo \
--namespace nebulaops-demo --create-namespace \
--set image.tag=1.0.0 \
--set replicaCount=2

# Upgrade (nuova versione)
helm upgrade nebulaops-demo ./helm/nebulaops-demo \
--namespace nebulaops-demo \
--set image.tag=1.1.0 \
--atomic # rollback automatico se upgrade fallisce
--timeout 120s

# Rollback alla versione precedente
helm rollback nebulaops-demo 1 --namespace nebulaops-demo

# Lista release con status
helm list -A

# Dry-run: renderizza template senza deployare
helm template nebulaops-demo ./helm/nebulaops-demo --dry-run

# Uninstall (rimuove tutte le risorse K8s)
helm uninstall nebulaops-demo --namespace nebulaops-demo

# Override values per ambiente produzione
helm upgrade nebulaops-demo ./helm/nebulaops-demo \
-f values-prod.yaml \
--set image.tag=$(git rev-parse --short HEAD)
```

CAPITOLO 6

ArgoCD — GitOps Continuo

Git come source of truth per il cluster Kubernetes

6.1 Il Paradigma GitOps

GitOps è un paradigma operativo in cui lo stato desiderato dell'infrastruttura è interamente descritto in un repository Git. Qualsiasi modifica — nuova versione dell'immagine, cambio di configurazione, aggiunta di un servizio — viene fatta tramite una commit o una pull request. Il sistema di deployment (ArgoCD) osserva continuamente il repository e riconcilia lo stato del cluster verso quello dichiarato in Git, garantendo convergenza automatica e audit trail completo.

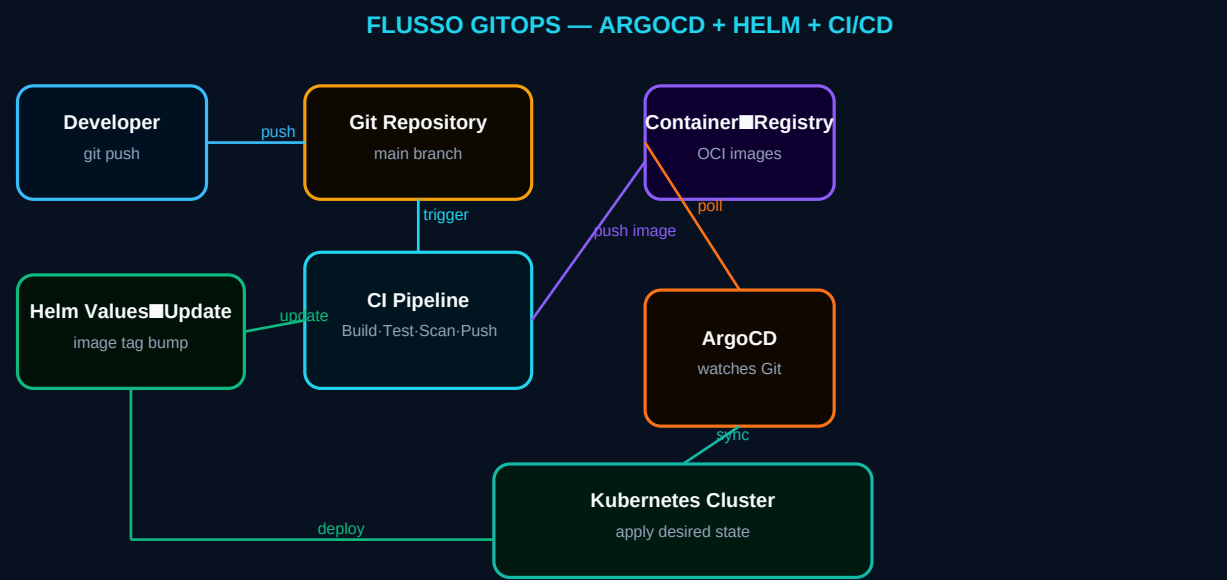


Fig. 6 — Flusso GitOps completo: Developer → Git → CI Pipeline → Registry → ArgoCD → Kubernetes

6.2 Concetti Fondamentali ArgoCD

Concetto	Definizione operativa
Application	Unità di deployment ArgoCD. Punta a un path Git (helm chart, kustomize, plain YAML) e a un namespace/cluster Kubernetes di destinazione.
Desired State	Stato definito in Git — i manifest YAML o il Helm chart che descrivono cosa dovrebbe girare nel cluster.
Live State	Stato attuale del cluster — cosa sta effettivamente girando al momento.
Sync Status	OutOfSync = differenza tra desired e live state. Synced = allineati.
Drift	Modifica manuale al cluster (kubectl apply diretto) che crea divergenza rispetto a Git. ArgoCD lo rileva e può riportare automaticamente allo stato Git (auto-heal).

Concetto	Definizione operativa
Health Status	Stato aggregato di salute dell'Application: Healthy, Degraded, Progressing, Missing, Unknown.
Auto-Sync	Se abilitato, ArgoCD applica automaticamente ogni commit su Git senza intervento manuale.
Sync Hooks	Hook pre/post-sync: database migration, smoke test, notifica Slack prima del deploy.
App of Apps	Pattern per gestire molte Application tramite una Application padre — utile per ambienti multi-tenant.

6.3 Installazione e Configurazione

```
# Installa ArgoCD nel cluster
kubectl create namespace argocd
kubectl apply -n argocd -f \
https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml

# Attendi che tutti i pod siano Running
kubectl wait --for=condition=Ready pod -l app.kubernetes.io/name=argocd-server \
-n argocd --timeout=120s

# Recupera la password admin iniziale
kubectl -n argocd get secret argocd-initial-admin-secret \
-o jsonpath="{.data.password}" | base64 -d

# Port-forward per accesso UI
kubectl port-forward svc/argocd-server -n argocd 8090:443
# → https://localhost:8090 (admin / )

# Crea Application per nebulaops-demo via CLI
argocd app create nebulaops-demo \
--repo https://github.com/yourorg/nebulaops-demo.git \
--path k8s \
--dest-server https://kubernetes.default.svc \
--dest-namespace nebulaops-demo \
--sync-policy automated \
--auto-prune \
--self-heal

# Sync manuale forzato
argocd app sync nebulaops-demo --force

# Status Application
argocd app get nebulaops-demo
```

CAPITOLO 7

Stack di Osservabilità

Prometheus · Loki · Tempo · Grafana · OpenTelemetry

7.1 I Tre Pilastri dell'Osservabilità

L'osservabilità moderna si basa su tre segnali distinti ma complementari: **metriche** (what is happening — numeri aggregati nel tempo), **log** (what happened — eventi discreti con contesto), **trace** (why it happened — percorso di una richiesta attraverso i servizi). NebulaOps implementa tutti e tre tramite tecnologie CNCF standard.

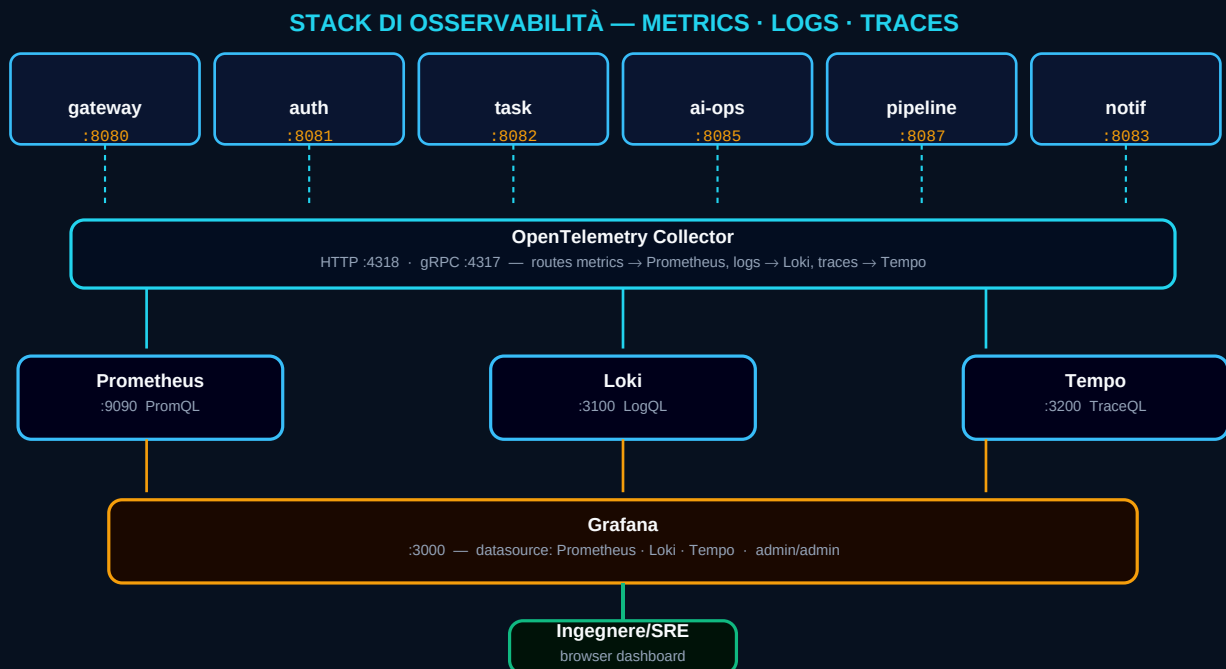


Fig. 7 — Stack osservabilità: microservizi → OTEL Collector → Prometheus/Loki/Tempo → Grafana unificato

7.2 Prometheus — Metriche Time-Series

Prometheus scrupa ogni microservizio Spring Boot ogni 15 secondi sull'endpoint `/actuator/prometheus`, che espone metriche in formato Exposition Text. Le metriche vengono archiviate localmente nel time-series database interno e interrogabili con PromQL (Prometheus Query Language).

```
# PromQL — Query utili

# HTTP request rate per servizio (ultimi 5 minuti)
rate(http_server_requests_seconds_count[5m])

# P95 latency per endpoint
histogram_quantile(0.95, rate(http_server_requests_seconds_bucket[5m]))

# Error rate (status 5xx)
```

```
rate(http_server_requests_seconds_count{status=~"5.."}[5m])
/ rate(http_server_requests_seconds_count[5m])

# JVM heap usage
jvm_memory_used_bytes{area="heap", job="gateway-service"}

# Container memoria Docker
container_memory_usage_bytes{name=~"nebulaops.*"}

# Accesso: http://localhost:9090
```

7.3 Loki — Aggregazione Log

Loki è il sistema di log aggregation ottimizzato per Kubernetes sviluppato da Grafana Labs. A differenza di ElasticSearch, **non indicizza il contenuto** dei log — solo le label (container, namespace, service). Questo lo rende significativamente più efficiente in termini di storage e CPU. I log vengono interrogati con LogQL.

```
# LogQL — Query utili (via Grafana Explore → Loki)

# Tutti i log del gateway con livello ERROR
{container="nebulaops-v21-2-1-gateway-service-1"} |= "ERROR"

# Log con JSON parsing e filtro per campo
{job="nebulaops"} | json | level="ERROR"

# Log con latenza > 2s
{job="nebulaops"} | json | duration > 2s

# Count errori per servizio negli ultimi 15 minuti
count_over_time({job="nebulaops"} |= "ERROR" [15m])

# Accesso: http://localhost:3100 (via Grafana)
```

7.4 Tempo — Distributed Tracing

Ogni richiesta HTTP che attraversa i microservizi genera uno **span** con un trace-id univoco. Questo permette di seguire la chiamata dal frontend attraverso gateway → task-service → MongoDB e vedere esattamente dove si è verificata una latenza o un errore. NebulaOps utilizza OpenTelemetry SDK integrato nei servizi Spring Boot (auto-instrumentation).

7.5 Grafana — Dashboard Unificata

Grafana (:3000, admin/admin) è il punto di analisi unificato. I tre datasource (Prometheus, Loki, Tempo) sono preconfigurati. La funzionalità più potente è la **correlazione dei segnali**: da un log con trace-id si può passare direttamente al trace Tempo corrispondente con un click, vedere i singoli span, e tornare alle metriche Prometheus dello stesso intervallo temporale.

URL / Datasource	Credenziali	Funzione
http://localhost:3000	admin / admin	Dashboard Grafana — entry point principale
http://localhost:9090	—	Prometheus UI — query PromQL dirette, targets

URL / Datasource	Credenziali	Funzione
http://localhost:3100	—	Loki (solo API, via Grafana Explore)
http://localhost:3200	—	Tempo (solo API, via Grafana Explore)
http://localhost:4318	—	OTel Collector HTTP endpoint (OTLP/HTTP)

CAPITOLO 8

CI/CD e Deploy su NebulaOps

Pipeline, strategie di deploy, progetto demo end-to-end

8.1 Pipeline GitHub Actions — 7 Stage

PIPELINE CI/CD — GITHUB ACTIONS / GITLAB CI

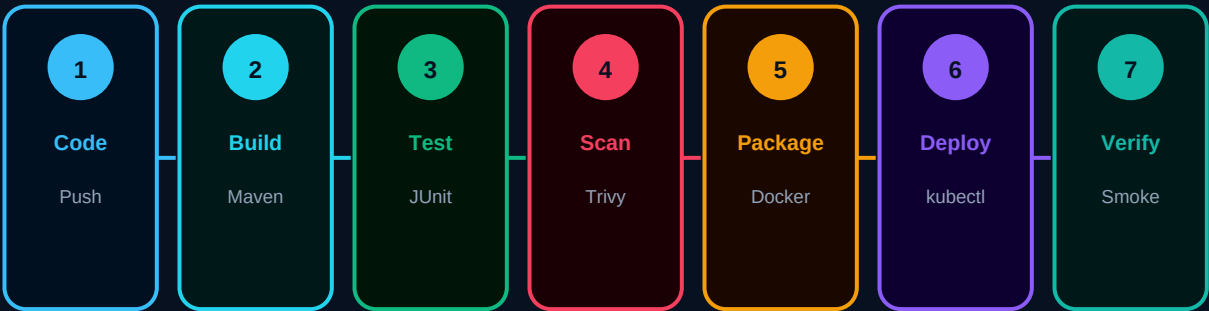


Fig. 8 — Pipeline CI/CD: Code Push → Build → Test → Security Scan → Package → Deploy → Verify

Stage	Tool	Durata tipica	Dettaglio
1. Code Push	Git / GitHub	< 1s	Trigger su push a main o PR. Workflow file: <code>.github/workflows/ci-cd.yml</code>
2. Build Maven	Maven 3.9 / Java 21	3-5 min	<code>mvn verify -DskipTests=false</code> . Layer cache GitHub Actions. Artifact: <code>target/*.jar</code>
3. Test JUnit	JUnit 5 / Mockito	1-2 min	Test unitari + integrazione. Report surefire upload come artifact anche in caso di fail.
4. Security Scan	Trivy	1-2 min	Scan immagine Docker per CVE. exit-code 1 se CRITICAL o HIGH trovati — blocca pipeline.
5. Package Docker	Docker Buildx	2-4 min	Multi-stage build. Push su registry con tag SHA commit + latest. Cache layer GHA.
6. Deploy kubectI	kubectI / Helm	< 1 min	<code>kubectI set image</code> per rolling update. Wait rollout status <code>--timeout=120s</code> .
7. Verify	curl / kubectI run	30s	Smoke test via pod temporaneo <code>curlimages/curl</code> . <code>curl -sf /api/health → exit 0</code> .

8.2 Strategie di Deploy

Strategia	Meccanismo	Quando usarla	Trade-off
Rolling Update (default)	Sostituisce pod uno alla volta. maxSurge:1, maxUnavailable:0.	Deploy standard. Zero downtime.	Versione vecchia e nuova coesistono brevemente.
Blue/Green	Due deployment identici. Switch del Service selector.	Rollback immediato. Testing pre-produzione.	Doppio uso di risorse durante la transizione.
Canary	5-10% traffico alla nuova versione prima del 100%.	Feature flags, A/B testing, alto rischio.	Richiede Ingress controller con pesi (Nginx, Istio).
Recreate	Down → Up: tutti i pod sostituiti simultaneamente.	Database schema migration obbligatoria.	Downtime accettato. Non usare in produzione.

8.3 Deploy su NebulaOps — nebulaops-demo

```
# === OPZIONE 1: Docker Compose integrato nello stack NebulaOps ===
# Aggiungere a docker-compose.yml del progetto nebulaops-v21.2:
nebulaops-demo:
  build: { context: ../nebulaops-demo-app, dockerfile: Dockerfile }
  image: nebulaops-demo:latest
  ports: ["8099:8099"]
  environment:
    MONGODB_URI: mongodb://nebula:nebula@mongodb:27017/nebulaops-demo?authSource=admin
    RABBITMQ_HOST: rabbitmq
  depends_on: {mongodb: {condition: service_healthy}}
  restart: unless-stopped

# Poi: docker compose up --build -d nebulaops-demo

# === OPZIONE 2: Kubernetes con Kind ===
kind create cluster --name nebulaops
docker build -t nebulaops-demo:latest nebulaops-demo-app/
kind load docker-image nebulaops-demo:latest --name nebulaops
kubectl apply -f nebulaops-demo-app/k8s/
kubectl rollout status deployment/nebulaops-demo -n nebulaops-demo

# === OPZIONE 3: Helm ===
helm install nebulaops-demo ./nebulaops-demo-app/helm/nebulaops-demo \
--namespace nebulaops-demo --create-namespace

# Verifica post-deploy
kubectl port-forward svc/nebulaops-demo 8099:80 -n nebulaops-demo &
curl http://localhost:8099/api/health

# → {"status":"UP","service":"nebulaops-demo","version":"1.0.0"}
```

CAPITOLO 9

DevSecOps e Sicurezza

Trivy, secret management, security context, compliance

9.1 Trivy — Vulnerability Scanning

Trivy (Aqua Security) è lo scanner di vulnerabilità open-source più usato nell'ecosistema Kubernetes. Analizza le immagini Docker per trovare CVE (Common Vulnerabilities and Exposures) nei package OS, nelle librerie applicative e nei file di configurazione. Il devsecops-service di NebulaOps esegue scan Trivy via Docker socket e mostra i risultati nella tab Security.

```
# Scan manuale di un'immagine
trivy image nebulaops-v21-2-1-gateway-service:latest

# Scan con output JSON strutturato
trivy image --format json --output scan-results.json myimage:tag

# In CI/CD: blocca se CVE CRITICAL o HIGH
trivy image --exit-code 1 --severity CRITICAL,HIGH myimage:tag

# Scan di un filesystem locale (per Terraform, IaC)
trivy fs --security-checks config,vuln ./terraform/

# Genera SBOM (Software Bill of Materials)
trivy image --format cyclonedx --output sbom.json myimage:tag
```

9.2 Security Context Kubernetes

```
# Esempio deployment con security context corretto
spec:
  template:
    spec:
      securityContext:
        runAsNonRoot: true # Non eseguire come root
        runAsUser: 1000 # UID specifico non-root
        fsGroup: 2000
      containers:
        - name: app
          securityContext:
            allowPrivilegeEscalation: false # Impedisce escalation
            readOnlyRootFilesystem: true # Filesystem read-only
          capabilities:
            drop: ["ALL"] # Rimuove tutte le capability Linux
```

CAPITOLO 10

Terraform e Infrastructure as Code

Gestione infrastruttura dichiarativa con cost estimation

10.1 Terraform Studio in NebulaOps

Il **terraform-studio-service** (:8096) è il layer di orchestrazione Terraform integrato in NebulaOps. Permette di eseguire terraform plan e terraform apply direttamente dall'interfaccia grafica, con visualizzazione del piano di modifica e stima del costo mensile prima dell'apply. La tab Terraform nella sidebar mostra i moduli attivi con stato e versione provider.

Funzionalità	Descrizione
Plan viewer	Mostra le risorse da creare/modificare/distruggere con diff colorato
Cost estimation	Stima mensile €/€ per ogni risorsa cloud pianificata (Infracost integration)
Apply workflow	Confirm → Apply con streaming output in real-time
State management	Visualizzazione terraform.tfstate corrente con list delle risorse
Module browser	Lista dei moduli Terraform disponibili con versione e provider
Variable editor	Form per impostare terraform.tfvars prima del plan

CAPITOLO 11

Troubleshooting e Problemi Noti

Diagnosi sistematica e soluzioni per i problemi più comuni

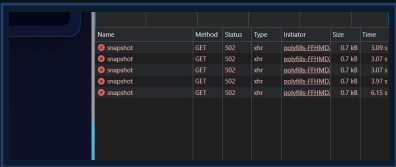


Fig. 9 — Errori 502 su /api/kubernetes/snapshot visibili nei Chrome DevTools (Network tab)

11.1 Errori 502 Bad Gateway

Il 502 indica che nginx frontend non riesce a raggiungere il gateway-service. È il problema più comune e ha cause ben definite in ordine di probabilità:

Causa	Diagnosi	Soluzione
Immagine gateway stale (causa #1)	docker compose ps → gateway-service Up ma vecchia immagine	./scripts/wsl/restart-gateway.sh (usa --no-cache sempre)
Gateway non avviato	docker compose ps → gateway-service Exiting o Restarting	docker compose logs gateway-service tail -50 verifica dipendenze MongoDB/RabbitMQ
OOMKill (out of memory)	docker inspect gateway-1 grep OOMKilled → true	Aumentare memoria WSL2 in .wslconfig a 16GB+ riavviare WSL: wsl --shutdown
Timeout kubectll (>30s)	curl -v http://localhost:8080/api/kubernetes/snapshot hangs per >15s	Normale senza cluster K8s. Verificare che Docker socket funzioni per il fallback.
Porta 8080 occupata	docker compose up → "bind: address already in use"	lsof -i :8080 → trovare PID → kill PID oppure cambiare porta in docker-compose.yml

```
# Diagnostica rapida 502
docker compose ps gateway-service
docker inspect nebulaops-v21-2-1-gateway-service-1 | python3 -c \
"import sys,json; c=json.load(sys.stdin)[0]; \
print(c['State']['Status'], 'OOM:', c['State']['OOMKilled'])"
docker logs nebulaops-v21-2-1-gateway-service-1 --tail=50 | grep -E
"ERROR|WARN|Exception"

# Fix universale
./scripts/wsl/restart-gateway.sh
```

11.2 Docker Desktop — Containers/Images/Volumes Vuoti

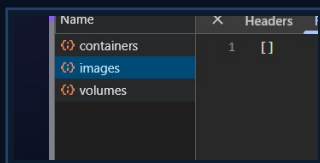


Fig. 10 — API /api/runtime/docker/containers restituisce [] — Docker socket non accessibile

Causa	Diagnosi	Soluzione
docker.sock non montato nel container gateway	docker exec gateway-1 ls -la /var/run/docker.sock → "No such file"	Verificare in docker-compose.yml: volumes: - /var/run/docker.sock:/var/run/docker.sock
Permessi docker.sock insufficienti	docker exec gateway-1 ls -la /var/run/docker.sock → srwxrwxrwx (no world-read)	Sul host WSL2: sudo chmod 666 /var/run/docker.sock Oppure: sudo usermod -aG docker USER
WSL2 Integration disabilitata	Docker Desktop → Settings → Resources → WSL Integration → Ubuntu OFF	Abilitare WSL Integration per Ubuntu Riavviare Docker Desktop

```
# Diagnosi docker socket
docker exec nebulaops-v21-2-1-gateway-service-1 \
ls -la /var/run/docker.sock
# → srwxrwxrwx = OK (world-writable)
# → No such file = volume non montato
# → Permission denied = permessi insufficienti

# Test diretto socket via curl Unix
curl --unix-socket /var/run/docker.sock http://localhost/containers/json | python3 -m json.tool | head -20
# Se OK sul host ma non nel container → WSL Integration issue
```

11.3 OpenLens — Sezioni Vuote (Pods, Deployments, ecc.)

Se tutte le sezioni OpenLens sono vuote, il gateway non riesce a costruire le risorse K8s. In assenza di kubectl, usa il Docker socket come fallback. Se anche Docker non funziona, tutte le sezioni restano vuote.

```
# Diagnosi snapshot Kubernetes
curl -s http://localhost:8080/api/kubernetes/snapshot | \
python3 -c "import sys,json; d=json.load(sys.stdin); \
print(f'resources: {len(d[chr(114)+chr(101)+chr(115)+chr(111)+chr(117)+chr(114)+chr(99)+chr(101)+chr(115)])}', \
f'live: {d[chr(108)+chr(105)+chr(118)+chr(101)]}')"
# Risposta attesa quando funziona:
```

```
# resources: 50 live: True

# Risposta quando Docker socket non funziona:
# resources: 0 live: False

# → seguire fix sezione 11.2 per Docker socket

# Con cluster Kubernetes reale
cp ~/.kube/config .kube/config
# Il docker-compose monta .kube/config:/kube/config:ro
# e imposta KUBECONFIG=/kube/config nel container gateway
```

11.4 MongoDB e RabbitMQ — Problemi di Avvio

Problema	Sintomo	Soluzione
MongoDB non diventa (healthy)	Dopo 2+ minuti ancora in "starting". docker compose ps → mongodb: starting	Volume corrotto: docker compose down -v (cancella dati!) docker compose up -d mongodb → attendere (healthy) prima di altri servizi
RabbitMQ restart loop	.erlang.cookie corrotto nel volume.	docker compose down rabbitmq docker volume rm nebulaops-v21-2-1_rabbitmq-data docker compose up -d rabbitmq
Redis connection refused	go-cache-service o altri servizi non riescono a connettersi a Redis.	docker compose restart redis Verificare: docker exec redis-1 redis-cli ping → PONG = OK
task-service non parte	Dipendenza da MongoDB e RabbitMQ. Esegue il retry connection per 3 minuti.	Attendere che mongodb e rabbitmq siano (healthy) prima che task-service si avvii. Il retry è automatico.

11.5 Performance e Gestione Memoria WSL2

■ ATTENZIONE

Lo stack completo NebulaOps usa ~6-8 GB di RAM. Su macchine con meno di 16 GB è necessario disabilitare i servizi non essenziali commentandoli in docker-compose.yml. Candidati alla disabilitazione: mongo-express, redis-commander, otel-collector, tempo, loki.

```
# Monitora uso memoria Docker in real-time
docker stats --format "table {{.Name}} {{.MemUsage}} {{.CPUPerc}}" \
$(docker ps --format "{{.Names}}")

# Verifica memoria disponibile WSL2
free -h

# Mem: used should be < total - 2GB

# Se WSL2 usa troppa memoria – libera la cache
sudo sh -c "echo 3 > /proc/sys/vm/drop_caches"

# Docker prune – rimuovi risorse non usate (immagini, container fermi, volumi dangling)
```

```
docker system prune -a --volumes
```

ATTENZIONE: rimuove anche i volumi dati! Usare solo se si può perdere i dati.

CAPITOLO 12

Riferimento API e Estensione della Piattaforma

Tutti gli endpoint, risposta attesa, come aggiungere un microservizio

12.1 Endpoint Gateway (:8080)

Metodo + Path	Parametri	Response shape	Note
GET /api/health	—	{status, service, version, ts}	Health check gateway
GET /api/ping	—	{ok, version, ts}	Ping rapido senza dipendenze
GET /api/kubernetes/snapshot	—	{cluster, resources[], logs[], live}	K8s o Docker-derived
GET /api/kubernetes/logs	ns? (query)	[[{time, service, level, message}]]	K8s events o Docker logs
GET /api/kubernetes/nodes	—	{live, tool, data}	kubectl get nodes -o json
GET /api/runtime/docker/containers	—	[[{id,name,image,status,ports,network}]]	Via Docker socket
GET /api/runtime/docker/images	—	[[{repository,tag,size,vulnerabilities}]]	Via Docker socket
GET /api/runtime/docker/volumes	—	[[{name,driver,mount,size}]]	Via Docker socket
GET /api/runtime/docker/networks	—	[[{Id,Name,Driver,...}]]	Via Docker socket
GET /api/runtime/helm/releases	ns? (query)	[[{name,namespace,chart,status}]]	helm list -A -o json
GET /api/platform/observability	—	{live, prometheus, loki, tempo, grafana}	Stack health
GET /api/platform/gitops	—	{live, applications[]}	ArgoCD applications
GET /api/platform/devsecops	—	{live, scans[], vulnerabilities[]}	Trivy results
GET /api/platform/environments	—	[[{name,namespace,status,resources}]]	K8s namespaces
POST /api/auth/login	body: {username,password}	{token, user}	Dev-mode auth

Metodo + Path	Parametri	Response shape	Note
POST /api/auth/register	body: {username,password,email}	{id, username}	Dev-mode register
GET /api/tasks	orgId? (query)	[[{id,title,status,priority,...}]]	Proxy → task-service
POST /api/tasks	body: {title,description,priority}	{id,...}	Crea task + evento MQ
PUT /api/tasks/{id}	body (parziale)	task aggiornato	Aggiorna task
DELETE /api/tasks/{id}	—	204 No Content	Elimina task
PATCH /api/tasks/{id}/status/{s}	—	task con nuovo status	Cambia stato
POST /api/ai-ops/analyze	body: {prompt}	{analysis,suggestions,confidence,live}	RCA AI
POST /api/ai-ops/autofix	body: {prompt}	{result,applied,live}	Auto-fix AI

12.2 API nebulaops-demo (:8099)

Metodo + Path	Body / Params	Response	Note
GET /api/health	—	{status, service, version, ts}	Spring Boot health
GET /api/items	status? (query)	[[{id,name,description,status,createdAt}]]	Filtra per status
GET /api/items/{id}	—	item o 404	Singolo item
POST /api/items	{name, description}	{id,...} 201	Crea + evento RabbitMQ
PUT /api/items/{id}	patch parziale	item aggiornato	Aggiorna + evento MQ
DELETE /api/items/{id}	—	204 No Content	Elimina + evento MQ
GET /actuator/health	—	{status, liveness, readiness}	K8s probe endpoint
GET /actuator/prometheus	—	Exposition Text	Scraping Prometheus

12.3 Aggiungere un Nuovo Microservizio

NebulaOps è progettato per l'estensione. Aggiungere un nuovo servizio richiede modifiche coerenti in cinque file — tutti coordinati da config/platform.yml.

Step	File da modificare	Cosa aggiungere
1	backend/nuovo-service/	Struttura Spring Boot: Dockerfile, pom.xml (parent: backend/pom.xml), application.yml con port, @RestController base
2	docker-compose.yml	Servizio con image, build context, ports, environment (MONGODB_URI, RABBITMQ_HOST...), depends_on
3	config/platform.yml	services: nuovo: { host: nuovo-service, port: 8099 }
4	backend/gateway-service/src/main/resources/application.yml	proxy: nuovo: \${NUOVO_SERVICE_URL:http://nuovo-service:8099}
5	backend/gateway-service/.../ProxyController.java	@Value("\${proxy.nuovo}") + @GetMapping/@PostMapping con logica forward e fallback
6	frontend/src/app/api.config.ts	nuovo: { list: `\${API_BASE}/nuovo` } nella struttura API
7	./scripts/wsl/restart-gateway.sh	Rebuild gateway — obbligatorio per caricare nuove @Value

```
# Esempio completo: aggiungere billing-service (:8097)

# 1. config/platform.yml
services:
billing: { host: billing-service, port: 8097 }

# 2. gateway application.yml
proxy:
billing: ${BILLING_SERVICE_URL:http://billing-service:8097}

# 3. ProxyController.java
@Value("${proxy.billing}")
private String billingUrl;

@GetMapping("/billing/invoices")
public ResponseEntity listInvoices() {
return forward(() -> rest.getForObject(billingUrl + "/api/invoices", Object.class),
List.of());
}

# 4. api.config.ts
billing: { invoices: `${API_BASE}/billing/invoices` }

# 5. Rebuild
./scripts/wsl/restart-gateway.sh
```